Khalani Whitepaper

Wojciech Klaudiusz Zaborowski
Wojciech Kołowski
Mateusz Pyzik

1 Intro

2 Cubix

3 The Axi programming language

Axi (pronounced "Axee") is the smart contract language of Khalani. It is a purely functional programming language with a strong, static, linear type system and support for formal reasoning about programs in classical higher-order logic. Among mainstream programming languages, it is most similar to Haskell and, to a lesser degree, OCaml. It was also heavily inspired by proof assistants based on Type Theory, like Coq, Lean and Agda.

3.1 Philosophy

Axi features a unique architecture in that it is split into two strictly separate layers: the programming layer and the logic layer.

The Programming Layer of Axi is a purely functional language based on System F with algebraic data types and a resource-aware type system. It aims to make programming pleasant and comprehending smart contract code easy, while providing essential guarantees to all on-chain code:

- **Type safety**. Well-typed programs can't go wrong, i.e. they can't crash (unless they run out of gas), can't perform silly operations like adding a number and a string and can't result in any other runtime type error.
- **Memory safety**. Well-typed programs can't result in memory errors, like writing to a random memory location, nor can they have memory leaks.
- **Resource safety**. Well-typed programs can't access resources they don't have permission to, can't delete tokens that are not supposed to be deletable and can't create money out of thin air.

The Logic Layer of Axi is a language for reasoning about Axi programs using classical higher-order logic. It allows expressing arbitrarily complex mathematical properties, thus giving programmers the ability to precisely specify the business logic and behaviour of their smart contracts. These specifications can then be either tested with random inputs before deployment, checked during runtime, or proved once and for all as theorems, thus providing guarantees

much more powerful than those of the type system alone. The use of the logic layer is entirely optional, i.e. programmers are never forced to engage with any part of it, although they will be encouraged to do so by competition from other programmers' code, which might become more popular if it provides stronger guarantees.

Strict separation. Logic can reason about programs, but programs cannot manipulate logical entities, like theorems or proofs, at runtime. This one-way interaction keeps the language relatively simple, while still allowing it to express almost all functional programming idioms and all possible mathematical properties and their proofs.

Guarantees all the way down. We intend not only for Axi to provide strong guarantees about its smart contracts, but also for us to provide strong guarantees about Axi itself. We plan to use the Coq proof assistant to formally verify significant portions of Axi's metatheory, including the aforementioned properties of type and resource safety, as well as the consistency of Axi's logic. We are also considering formal verification of the reference implementation of Axi. The goal of these efforts is to ensure that the guarantees that Axi provides are trustworthy and will stay so forever, as long as the consensus protocol is followed.

3.2 Features

In this section, we will take a closer look at Axi's language features.

3.2.1 Programming Layer

Axi is, at its core, a purely functional programming language descending from the ML family. Thus, it supports all the usual features of such languages:

- **Purity amd immutability**. In Axi, all values are pure and immutable – they cannot be null, cannot throw exceptions and once created, they cannot be changed. This facilitates reasoning about and comprehending code, thus rendering Axi smart contracts transparent and auditable.

- **First-class, higher-order and anonymous functions**. Functions are first class citizens. They can be passed as arguments to other functions, returned from other functions as results and created on the fly. Axi encourages a programming style in which data is processed by pipelines built from basic building blocks like mapping, reducing, filtering and so on.

- **Algebraic data types and exhaustive pattern-matching**. Instead of clunky boolean flags and cascades of ifs, Axi makes it easy to model the problem domain with disjoint unions and define functions on them by pattern matching and explicitly handling all possible cases.

- **Strong static type system**. Axi's type system makes it easy to work with high-level abstractions, like merkle trees or state machines, instead of low-level primitives, like integers or byte arrays. It is lightweight, not placing too many demands on the programmer, but still catches the most egregious errors – it can easily protect programmers from confusing with each other such objects as an integer, a string, an address, a transaction hash, a token amount and a cryptographic secret.

- **Polymorphism and interfaces**. Axi's type system features polymorphism (also known as "generics") as well as interfaces inspired by Haskell's type classes. These features make it easy to implement data structures and functions that work for all types that support a certain behaviour, for example a hash map that works for all types of ERC-20-compliant tokens bridged from an external blockchain.

However, these are only the more or less standard features that plenty of functional programming languages have. The killer feature, as far as the programming layer is concerned, is tracking correct resource usage at the type level. This aspect of Axi's type system was inspired by linear logic and its cousin, Quantitative Type Theory, and improves upon Move, the programming language of the Sui and Aptos blockchains.

In an Axi program, every variable is assigned its *quantity*, which can be either: zero, one, at most one, at least one, unrestricted. The variable's quantity determines how many times it can be used in the program, with a type error being produced if it was used too much or not enough. For example, the typical variable holding an NFT will have a quantity of one, forcing the program to do something with the NFT before it terminates: maybe give the NFT to the transaction's originator, maybe trade it for something else (and handle that something else properly according to its quantity), or maybe send it to somebody as a gift. What could not be done in such a situation is forgetting the NFT and doing nothing with it, effectively making it "lost" in the guts of the chain.

3.2.2 Logic Layer

Axi's logic layer consists of (typed) classical higher-order logic with equality whose domains of discourse are the types from the programming layer. This means the logic consists of:

- **Propositional connectives and constants**. Standard propositional connectives ($\implies$, $\wedge$, $\vee$, $\iff$, $\sim$) and constants ($\top$, $\bot$) can be used to build up complex propositions from simpler ones.

- **First-order quantification**. Axi supports quantification, both universal (for all x, ...) and existential (there exists an x such that...). We can quantify over terms (forall all x of type A, ...) as well as over types themselves (forall all types A, ...). This means we can freely talk about data, resources and programs alike, as well as about data types, resource types and program types.

- **Higher-order quantification**. We can also quantify over logical entities like propositions, predicates and relations. This is precisely what it means for a logic to be higher-order.

- **Equality**. The built-in equality relation (=) makes it possible to precisely reason about programs' input-output behaviour.

- **Classical principles**. Unlike many other program logics, we assume the Law of Excluded Middle as well as the Axiom of Choice, which makes the logic classical and thus intimately familiar to all mathematicians.

Proofs are carried out in a proof language based on the Curry-Howard correspondence: proofs of implications and universal quantifiers are like functions, proofs of existential quantifiers and conjunctions are like pairs, proofs of disjunctions are like disjoint union constructors, etc. Negation and biconditional are handled as in intuitionistic logic - instead of being primitives, negation is a shorthand for an implication into falsehood, whereas biconditional is a shorthand for a conjunction of two implications.

One novel feature that distinguishes our proof language is that the proofs can be presented in two styles: either in a *proofterm style*, similar to the style of Agda and Idris (as well as Coq's Gallina terms and Lean's terms), or in a *tactic style*, similar to the way proofs usually look in Coq or Lean. This feature makes it easy to accomodate people coming from all these languages. How this works is pretty simple: Axi's proofs are actually proof terms (like Agda/Idris), but thanks to a very clever parser they can be formatted either like terms or like tactics.

Equality is handled as in Martin-Löf Type Theory - proved by reflexivity and eliminated with the so-called J rule, which is a principle that allows to substitute equals for equals in the remaining part of the proof. Because proofs are hard to write using these primitives, we provide a very convenient feature called *chaining*, which allows to perform reasoning with long chains of equations more like it's usually done on the blackboard. As this feature is very convenient, it also supports implications, biconditionals and other transitive relations.

Classical logic is realized using two primitives. Proof by contradiction allows to prove $((\sim P \implies \bot) \implies P)$ for any proposition P. The Axiom of Choice, realized in a way inspired by the Lean proof assistant, works in a way that might seem strange to mathematicians at a first glance, but makes it very convenient to use in practice: while writing programs for the purpose of using it in a proof, we are allowed make use of a new primitive, `choose`, which turns a proof of $\exists x : A, P\ x$ into an element of type A. These programs that make use of the Axiom of Choice cannot be run, but since they sole purpose is to be used as part of a proof, this does not pose any problems.

Last but not least, the killer feature of the logic layer are *rich interfaces*. They are very much like the interfaces available in the programming layer, but in addition to function declarations they may also contain logical specifications. When implementing such an interface, besides defining the functions, we must also prove the specifications. TODO

3.3 Interacting with the blockchain

3.3.1 Primitives

```
primitive
  // provides access to ledger and transaction metadata
  Ledger : Type1
  // each cell and user has a unique address
  Address : Type
  // an address that signed the currently executed transaction
  whoami : Ledger -> Address * Ledger
  // ‘transfer‘ creates a new owned cell, owned by a specified address
  transfer : forall (A : Type1), Address -> A -> Ledger -> Address *
    Ledger
  // ‘receive‘ destructs an existing cell owned by transaction signer
```

```
  receive : forall @(A : Type1), Ledger -> Maybe A * Ledger
  // ‘share‘ creates a new mutable, shared cell, accessible by everyone
  share : forall (A : Type1), A -> Ledger -> Address * Ledger
  // ‘update‘ accesses an owned or shared cell but requires a new
    version back
  update : forall @(A : Type1), Address -> Ledger -> Maybe (A * (A ->
    Unit)) * Ledger
  // ‘freeze‘ creates a new immutable cell, accessible by everyone
  freeze : forall (A : Type), A -> Ledger -> Address * Ledger
  // ‘peek‘ provides contents of an immutable object
  peek : forall @(A : Type), Address -> Ledger -> Maybe A * Ledger
```

3.3.2 Transaction execution

Combining Khalani's ledger model with Axi's elegant functional programming and powerful resource usage tracking at the type level results in a pleasant programming model characterized by a very high standard of safety and correctness. The typical program executed as part of a transaction will interact with the blockchain in three steps:

- **Withdraw**. First, resources are "withdrawn" from the blockchain state into the program. The ledger's ownership model ensures the safety of this step, for example by making sure that the program cannot withdraw resources that don't belong to the transaction's originator.
- **Transform**. Second, the program transforms its resources by performing various computations, exchanging the resources for other resources and moving them around, etc. The safety of this step is guaranteed by the type system. The resources cannot get unlawfully destroyed or copied because the type system keeps an eye on them, silly operations cannot be performed on them, because the program is well-typed, they cannot get lost in the jungle of pattern matching, because exhaustiveness is checked, and so on.
- **Deposit**. Third, the program "deposits" the transformed resources back onto the blockchain. Again, the safety of this step is ensured by the ledger's ownership model.

3.4 Examples

Example 1 (Donut Shop)**.** *Click to see the donut shop example on Notion*

```
primitive
  Ledger : Type1
  Address : Type
  whoami : Ledger -> Address * Ledger
  transfer : forall (A : Type1), Address -> A -> Ledger -> Address *
    Ledger
  share : forall (A : Type1), A -> Ledger -> Address * Ledger
  halt : Ledger -> Ledger

interface Coin (A : Type1) where
  CoinAmount : Type
  coinAmount : A -> CoinAmount * A
  coinZero : A
  coinMerge : A -> A -> A
  coinSplit : CoinAmount -> A -> Option A * A

```

```
record type KhalaniCoin : Type1 where
  amount : Nat

instance Coin KhalaniCoin where
  CoinAmount = Nat
  coinAmount (coin : KhalaniCoin) : Nat * KhalaniCoin = (coin.amount, coin)
  coinZero = record where amount = 0
  coinMerge (coin1 coin2 : KhalaniCoin) : KhalaniCoin =
    record where amount = coin1.amount + coin2.amount
  coinSplit (amount : Nat) (coin : KhalaniCoin) : Option KhalaniCoin * KhalaniCoin =
    if amount <= coin.amount then
      (some (record where amount = amount), record where amount = coin.amount - amount)
    else
      (none, coin)

// Donut - simple purchasable object
record type Donut : Type1 where
  serialNumber : Nat

// Donut shop - shared object
record type DonutShop : Type1 where
  owner : Address
  price : Nat
  balance : KhalaniCoin
  nextSerialNumber : Nat

// Create a donut shop
init (ledger : Ledger) : Ledger =
  let (sender, ledger) = whoami ledger in
  let shop : DonutShop = record where
    owner = sender
    price = 1000
    balance = coinZero
    nextSerialNumber = 0 in
  let (_shopId, ledger) = share shop ledger in
  ledger

// Anybody can buy a donut for a set price
buyDonut (payment : KhalaniCoin) (shop : DonutShop) (ledger : Ledger) : KhalaniCoin * DonutShop * Ledger =
  let (splitResult, change) = coinSplit shop.price payment in
  match splitResult with
  | none => (change, shop, halt ledger)
  | some paid =>
    let newBalance = coinMerge shop.balance paid in
    let newSerialNumber = shop.nextSerialNumber in
    let newShop : DonutShop =
      record shop where
        balance = newBalance
        nextSerialNumber = newSerialNumber + 1 in
    let (sender, ledger) = whoami ledger in
    let donut : Donut = record where serialNumber = newSerialNumber in
    (change, newShop, transfer sender donut ledger)

// Consume donut and get nothing useful in return
eatDonut (donut : Donut) : Unit =
  let record {} = donut in ()

// Collects profits from shop (succeeds only for owner)
```

```
collectProfits (shop : DonutShop) (ledger : Ledger) : DonutShop *
    Ledger =
  let (sender, ledger) = whoami ledger in
  if shop.owner == sender then
    let profits = shop.balance in
    let newShop = record shop with balance = coinZero in
    let (_profitsId, ledger) = transfer sender profits ledger in
    (newShop, ownerCap, ledger)
  else
    (shop, ownerCap, ledger)
```

3.5 Formal description

Click to see slides with the theory

4 Outro